CONTACT MANAGEMENT SYSTEM

Using Circular Doubly Linked List, Array and Queue

Data Structures and Algorithms (DSA) Mini-Project Report

1. Problem Statement

Develop a contact management system in C++ using a circular doubly linked list, an array and a queue. Contact details (ID, name, phone, email, slot number) are stored in the circular list. Slots are managed using an array. Deleted contacts are held in a queue, and the oldest can be restored. The system supports adding, deleting, searching, forward and backward display, slot status and history.

2. Objective

The objective of this project is to develop a Contact Management System in C++ using structures, without classes. Contact details such as ID, name, phone, email and allotted slot number are stored in a circular doubly linked list, which allows easy insertion, deletion, searching and traversal in both forward and backward directions. Contact slots are managed using an array so a free slot can be allotted to each new contact. Deleted contacts are stored in a queue, which keeps the delete history and lets the oldest deleted contact be restored. The project applies circular linked list, array and queue concepts to solve a real-world contact management problem efficiently.

3. Data Structures Used

Data Structure	Used For
Circular Doubly Linked List	Stores contact records (forward and backward traversal; tail = head->prev)
Array (10 slots)	Slot allocation and availability (0 = free, 1 = occupied)
Queue (circular array, 5 records)	Delete history (FIFO) and restoring the oldest deleted contact

Node structure

Each node of the circular linked list stores a contact record (contact ID, name, phone number, email and allotted slot number) together with a *prev* pointer and a *next* pointer.

4. Algorithm

Main

Step 1: START. Create an empty circular doubly linked list, an array of 10 free slots and an empty queue.

Step 2: Display the menu: Add, Delete, Search, Display forward, Display backward, Slot status, Undo delete, Delete history, Exit.

Step 3: Read the choice and call the matching function.

Step 4: Repeat Steps 2 and 3 until choice = 0. Free all nodes of the list. STOP.

Add Contact

Step 5: Find the first free slot in the array. If none is free, display "No slots available" and stop.

Step 6: Read contact ID. If the ID already exists in the list, stop.

Step 7: Read name, phone and email.

Step 8: Create a new node and insert it at the end of the list (link it between tail and head and update the prev/next links; if the list is empty, the node points to itself and becomes head).

Step 9: Mark the slot as occupied and increase count.

Delete Contact

Step 10: Read contact ID and search the list from head until head is reached again. If not found, display "Contact not found" and stop.

Step 11: If the queue is full, remove the oldest record from the front of the queue.

Step 12: Insert the contact record at the rear of the queue (rear = (rear + 1) % size, queue count = queue count + 1).

Step 13: Free the contact's slot in the array.

Step 14: Delete the node by linking its previous and next nodes (update head if needed; if it is the only node, head = NULL).

Step 15: Decrease count.

Search Contact

Step 16: Read the contact ID.

Step 17: Traverse the list from head using next until head is reached again.

Step 18: If a node matches, display the contact details; otherwise display "Contact not found".

Display Contacts

Step 19: Forward: start at head and print each node using next until head is reached again.

Step 20: Backward: start at tail (head->prev) and print each node using prev until tail is reached again.

Undo Last Delete

Step 21: If the queue is empty, display a message and stop.

Step 22: Check the record at the front of the queue. If its ID is already in the list or no slot is free, display a message and stop.

Step 23: Remove the front record from the queue (front = (front + 1) % size, queue count = queue count - 1).

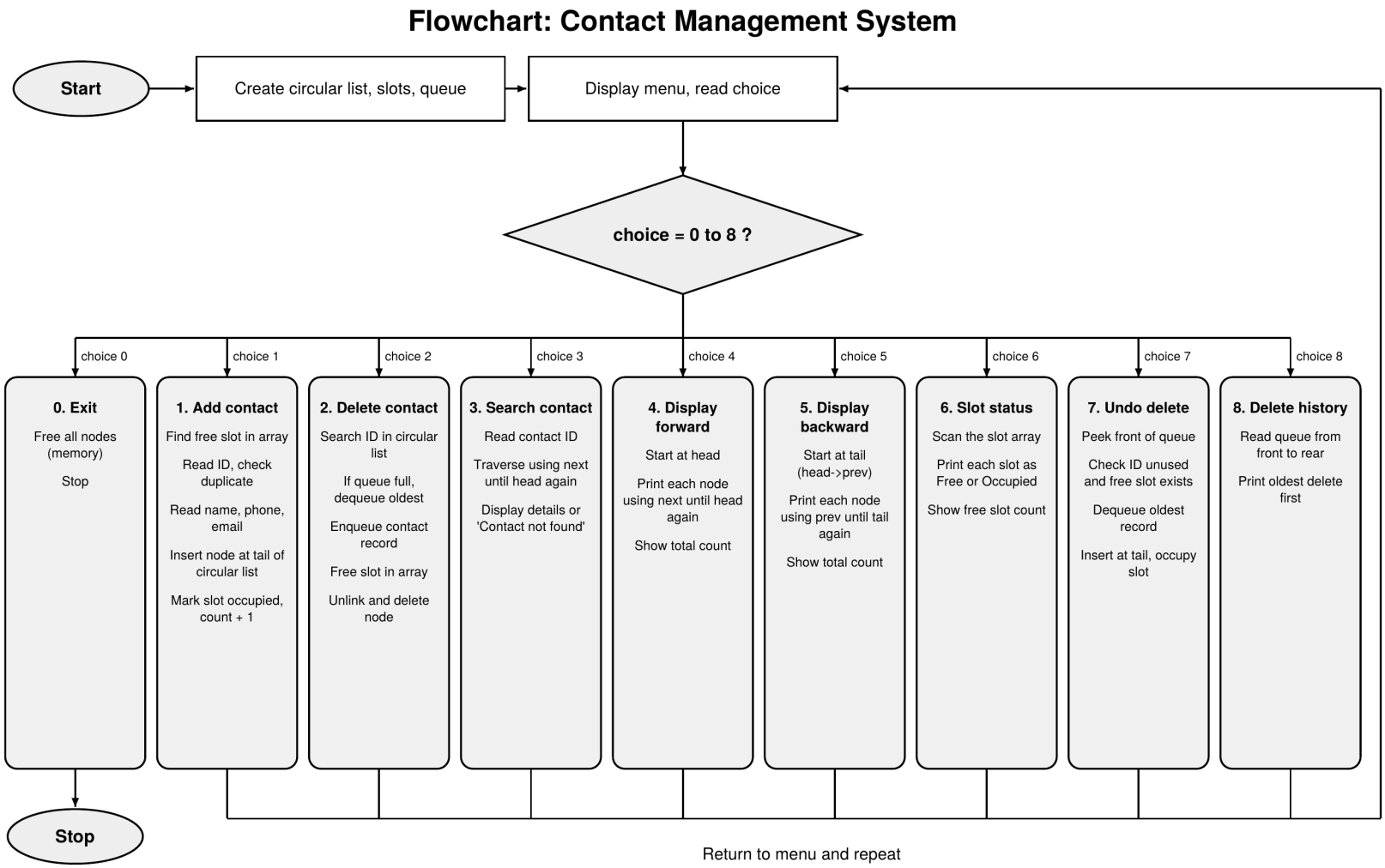
Step 24: Insert the contact at the end of the list and mark a free slot as occupied.

Slot Status and Delete History

Step 25: Slot status: traverse the slot array and print each slot as Free or Occupied.

Step 26: History: print the queue records from front to rear (oldest delete first).

5. Flowchart


Flowchart: Contact Management System
Start
Create circular list, slots, queue
Display menu, read choice
choice = 0 to 8 ?
choice 0
choice 1
choice 2
choice 3
choice 4
choice 5
choice 6
choice 7
choice 8
0. Exit
Free all nodes (memory)
Stop
1. Add contact
Find free slot in array
Read ID, check duplicate
Read name, phone, email
Insert node at tail of circular list
Mark slot occupied, count + 1
2. Delete contact
Search ID in circular list
If queue full, dequeue oldest
Enqueue contact record
Free slot in array
Unlink and delete node
3. Search contact
Read contact ID
Traverse using next until head again
Display details or 'Contact not found'
4. Display forward
Start at head
Print each node using next until head again
Show total count
5. Display backward
Start at tail (head->prev)
Print each node using prev until tail again
Show total count
6. Slot status
Scan the slot array
Print each slot as Free or Occupied
Show free slot count
7. Undo delete
Peek front of queue
Check ID unused and free slot exists
Dequeue oldest record
Insert at tail, occupy slot
8. Delete history
Read queue from front to rear
Print oldest delete first
Stop
Return to menu and repeat

6. Menu Options

Choice	Operation
1	Add contact
2	Delete contact
3	Search contact
4	Display contacts (forward)
5	Display contacts (backward)
6	Show slot status
7	Undo delete (restore oldest deleted contact)
8	Show delete history
0	Exit

7. Data Structure Operations

Operation	Description	Time Complexity
Add	Adds a contact at the tail of the list and occupies a slot	O(n)
Delete	Enqueues record, frees slot, removes node	O(n)
Searching	Finds a contact by ID	O(n)
Forward Traversal	Displays records from head to tail	O(n)
Backward Traversal	Displays records from tail to head	O(n)
Slot Status	Scans the slot array	O(1)
Undo Delete	Dequeues the oldest record and re-adds the contact	O(n)
Delete History	Prints the queue from front to rear	O(1)

8. Advantages

- Supports traversal in both directions, and the circular links make the tail reachable as head->prev.
- Allows dynamic addition and removal of records.
- Deletion is efficient once the node is found, since both neighbours are linked.
- Slots are allocated and freed automatically using the array.
- The queue keeps a FIFO delete history and restores the oldest deleted contact.
- Demonstrates practical use of three data structures in a contact book scenario.

9. Limitations

- Records are lost when the program terminates, as file storage is not implemented.

- The contact book has a fixed number of slots (10) and the delete history holds only 5 records.
- When the queue is full, the oldest history record is removed to make room.
- Searching for a contact requires linear traversal.
- The project is intended for educational purposes and is not a production-ready contact application.

10. Conclusion

The Contact Management System shows how a circular doubly linked list, an array and a queue can work together in one menu-driven C++ program. The list stores and traverses the contact records in both directions, the array allots and frees contact slots, and the queue keeps the delete history so that the oldest deleted contact can be restored. The project helps in understanding pointers, dynamic memory allocation and the practical use of basic data structures.